

Curso Profesional de Entity Framework Core 8

Módulo 2 - Modelado de datos con Entity Framework Core

PRÁCTICAS

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 2.1 - Convenciones de modelado en Entity Framework Core	3
Punto 2.2 - Entidades y propiedades	7
Punto 2.3 - Relaciones uno a muchos	11
Punto 2.4 - Relaciones uno a uno	16
Punto 2.5 - Relaciones muchos a muchos	22
Punto 2.6 - Configuración mediante Data Annotations	28
Punto 2.7 - Configuración mediante Fluent API	34
Punto 2.8 - Claves primarias, alternativas y compuestas	41
Punto 2.9 - Índices y restricciones	47
Punto 2.10 - Filtros globales de consulta	54
Punto 2.11 - Soft Delete	61
Punto 2.12 - Clean Architecture y Arquitectura Hexagonal	68

Módulo 2 - Prácticas de modelado de datos con Entity Framework Core 8

Estas prácticas continúan exactamente desde `M01/PROYECTO/1.12`. Cada punto dispone de una solución autónoma y completa en `M02/PROYECTO/2.x/AceriaData.sln`. El estado de un punto se construye sobre el punto anterior y las migraciones evolucionan el mismo esquema de `AceriaDB`.

Punto 2.1 - Convenciones de modelado en Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.1 parte de `M01/PROYECTO/1.12` y tiene como objetivo inspeccionar el modelo heredado sin modificar todavía su estructura.

Objetivos de aprendizaje

- Reconocer qué descubre EF Core por convención.
- Inspeccionar entidades, tablas, claves primarias y claves foráneas.
- Comprobar las navegaciones ya existentes.
- Diferenciar convención de configuración explícita.
- Trabajar con el ServiceProvider local sin campos estáticos.
- Conservar intacto el esquema heredado de M1.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.1
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en `M01/PROYECTO/1.12`. En 2.1 se introduce exclusivamente el contenido que corresponde a **Convenciones de modelado en Entity Framework Core**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();

foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
{
    var pk = entity.FindPrimaryKey();
    global::System.Console.WriteLine(
        $"Entidad: {entity.ClrType.Name} | Tabla: {entity.GetTableName()} | " +
        $"PK: {string.Join(", ", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}");
}
```

CSHARP

Línea 1	<code>using var provider = services.BuildServiceProvider(...);</code> → construye el contenedor ya configurado en el estado heredado de M1.
Línea 2	<code>using var scope = provider.CreateScope();</code> → crea un ámbito válido para resolver servicios scoped.
Línea 3	<code>GetRequiredService<AceriaDbContext>()</code> → obtiene el DbContext sin guardar el proveedor en un campo estático.
Línea 4	<code>context.Model.GetEntityTypes()</code> → recorre el modelo que EF Core ha construido realmente.
Línea 5	<code>entity.FindPrimaryKey()</code> → permite comprobar qué propiedad o propiedades forman la clave primaria.

Paso 4: Confirmar que 2.1 no crea una migración

Este punto no cambia el modelo. Se conservan las migraciones heredadas de M1 y **no** se ejecuta `EnsureCreated()`. La práctica se limita a inspeccionar metadatos.

```
dotnet ef migrations list
```

POWERSHELL

El resultado debe mostrar el historial heredado, sin una migración nueva de 2.1.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia `--- MODELO EF CORE 2.1 ---`. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Se crea un ServiceProvider estático	Se copia una versión antigua del ejercicio	Usar el provider local de Main y CreateScope.
Se recrea la base	Se usa EnsureCreated/EnsureDeleted innecesariamente	2.1 sólo inspecciona metadatos; no cambia el esquema.
Se introduce una relación nueva	Se adelanta contenido	Reservar 1:1 para 2.4 y N:M para 2.5.

Reto resuelto

Reto: Enumerar las claves foráneas de cada entidad y mostrar también DeleteBehavior sin modificar el modelo.

Solución: partir del código de `M02/PROYECTO/2.1`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Inspeccionar el modelo es leer el plano de una instalación antes de modificarla: primero se identifica qué está ya construido.

Resultado esperado

Al terminar 2.1, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene `--- MODELO EF CORE 2.1 ---`. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.2. Se parte del proyecto completo de 2.1; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.1

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
}
```

```

OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
void Agregar(OrdenFabricacion orden);
void Eliminar(OrdenFabricacion orden);
int Guardar();
}

public sealed class OrdenRepositorio : IOOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        global::System.Console.WriteLine("--- MODELO EF CORE 2.1 ---");
        foreach (var entity in context.Model.GetEntityTypes().OrderBy(e => e.ClrType.Name))
        {
            var pk = entity.FindPrimaryKey();
            global::System.Console.WriteLine($"Entidad: {entity.ClrType.Name} | Tabla: {entity.GetTableName()} | PK:

```

```
{string.Join(",", pk?.Properties.Select(x => x.Name) ?? Array.Empty<string>())}");
    foreach (var fk in entity.GetForeignKeys())
    {
        global::System.Console.WriteLine($" FK: {string.Join(",", fk.Properties.Select(x => x.Name))} ->
{fk.PrincipalEntityType.ClrType.Name}");
    }
}
}
```

Punto 2.2 - Entidades y propiedades

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.2 parte de 2.1 y tiene como objetivo enriquecer las entidades y configurar tipos, longitudes, requeridos, precisión y valores por defecto.

Objetivos de aprendizaje

- Añadir propiedades de negocio al modelo existente.
- Distinguir propiedades requeridas y opcionales.
- Configurar longitudes máximas.
- Configurar precisión decimal.
- Configurar valores por defecto.
- Generar una migración acumulativa sin introducir todavía índices del punto 2.9.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.2
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.1. En 2.2 se introduce exclusivamente el contenido que corresponde a **Entidades y propiedades**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.Property(x => x.Peso).HasPrecision(18, 3);
    entity.Property(x => x.Activa).HasDefaultValue(true);
});
```

CSHARP

Línea 1	DateTime? FechaEntrega → declara una fecha opcional; la columna puede admitir NULL.
Línea 2	string? Observaciones → declara texto opcional.

Línea 3	HasPrecision(18, 3) → fija precisión y escala del peso.
Línea 4	HasDefaultValue(true) → establece el valor por defecto de Activa en la base de datos.
Línea 5	HasMaxLength(...) → evita que las cadenas queden como nvarchar(max) cuando el dominio conoce su tamaño.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_2
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia **2.2 OK**. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
La migración contiene índices de negocio	Se adelantó contenido de 2.9	En 2.2 configurar sólo propiedades; la unicidad se formaliza después.
FechaEntrega no admite NULL	Se declaró DateTime en vez de DateTimeOffset	Usar DateTimeOffset?
Peso pierde escala	Se dejó el mapping por defecto	Configurar HasPrecision(18,3).

Reto resuelto

Reto: Añadir y comprobar una propiedad opcional Observaciones y una propiedad decimal Peso con escala 3, generando la migración acumulativa.

Solución: partir del código de **M02/PROYECTO/2.2**, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Configurar propiedades equivale a fijar tolerancias dimensionales y formatos antes de fabricar una pieza.

Resultado esperado

Al terminar 2.2, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.2 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.3. Se parte del proyecto completo de 2.2; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.2

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
}
```

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
        entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
        entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
        entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
        entity.Property(o => o.Observaciones).HasMaxLength(500);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
}

}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
    }
}

```

```

        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
        };

        context.OrdenesFabricacion.Add(orden);
        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
        context.SaveChanges();

        global::System.Console.WriteLine($"2.2 OK | Órdenes: {context.OrdenesFabricacion.Count()} | Planchas:
{context.PlanchasAcero.Count()}");
    }
}

```

Punto 2.3 - Relaciones uno a muchos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.3 parte de 2.2 y tiene como objetivo hacer explícita la relación OrdenFabricacion 1 -> N PlanchaAcero.

Objetivos de aprendizaje

- Identificar principal y dependiente.
- Configurar HasOne/WithMany.
- Definir la clave foránea OrdenId.
- Configurar DeleteBehavior.Cascade.
- Cargar la colección con Include.
- No adelantar relaciones uno-a-uno ni muchos-a-muchos.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.3
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.2. En 2.3 se introduce exclusivamente el contenido que corresponde a **Relaciones uno a muchos**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

CSHARP

Línea 1	<code>HasOne(x => x.Orden)</code> → selecciona la navegación de referencia del dependiente.
Línea 2	<code>WithMany(o => o.Planchas)</code> → selecciona la colección del principal.
Línea 3	<code>HasForeignKey(x => x.OrdenId)</code> → declara la FK de <code>PlanchaAcero</code> .
Línea 4	<code>OnDelete(DeleteBehavior.Cascade)</code> → define el comportamiento cuando se elimina la orden.
Línea 5	<code>IsRequired()</code> → confirma que una plancha persistida debe pertenecer a una orden.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_3
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.3 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Include no carga planchas	Se olvidó Include	Usar Include(o => o.Planchas).
FK inválida	La plancha apunta a una orden inexistente	Insertar el agregado o asignar una OrdenId válida.
Se añade OrdenAleacion	Se adelantó 2.5	Mantener 2.3 exclusivamente en la relación 1:N.

Reto resuelto

Reto: Crear una orden con dos planchas, recuperarla con Include y comprobar que la colección contiene exactamente dos elementos.

Solución: partir del código de M02/PROYECTO/2.3, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Una relación uno-a-muchos se parece a una orden de producción que agrupa varias planchas, mientras cada plancha pertenece a una sola orden.

Resultado esperado

Al terminar 2.3, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.3 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.4. Se parte del proyecto completo de 2.3; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.3

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;
```

CSHARP

```

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
        });
    }
}

```

```

});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
}
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
    }
}

```

```

        .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
};

context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var cargada = context.OrdenesFabricacion.Include(o => o.Planchas).Single(o => o.NumeroOrden == "OF-M2-0001");
global::System.Console.WriteLine($"2.3 OK | {cargada.NumeroOrden} | Planchas: {cargada.Planchas.Count}");
}
}

```

Punto 2.4 - Relaciones uno a uno

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.4 parte de 2.3 y tiene como objetivo añadir DetalleOrden y CertificadoCalidad como relaciones uno-a-uno.

Objetivos de aprendizaje

- Modelar relaciones uno-a-uno.
- Elegir el dependiente mediante HasForeignKey<T>.
- Mantener OrdenId obligatorio en el dependiente.
- Permitir que una orden exista sin detalle mediante navegación nullable.
- Configurar la relación una sola vez.
- Comprobar índices únicos de las claves foráneas.

Paso 1: Abrir la solución autónoma del punto

POWERSHELL

```

cd M02/PROYECTO/2.4
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release

```

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.3. En 2.4 se introduce exclusivamente el contenido que corresponde a **Relaciones uno a uno**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);

    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

CSHARP

Línea 1	<code>HasOne(d => d.Orden)</code> → configura la relación desde el dependiente.
Línea 2	<code>WithOne(o => o.Detalle)</code> → indica cardinalidad uno-a-uno.
Línea 3	<code>HasForeignKey<DetalleOrden>(d => d.OrdenId)</code> → identifica explícitamente al dependiente y su FK.
Línea 4	<code>IsRequired()</code> → hace obligatorio <code>OrdenId</code> para un <code>DetalleOrden</code> existente.
Línea 5	<code>DetalleOrden? Detalle</code> → permite, al mismo tiempo, que una orden todavía no tenga detalle.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_4
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.4 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Configuración contradictoria	Se configura la misma relación con <code>IsRequired</code> y <code>IsRequired(false)</code>	Configurar una sola vez desde <code>DetalleOrden</code> .
Dos detalles para la misma orden	La FK tiene índice único	Crear como máximo un dependiente por orden.
Detalle sin orden	<code>OrdenId</code> no apunta a una fila válida	Guardar la orden o asociar correctamente la navegación.

Reto resuelto

Reto: Crear un `CertificadoCalidad` para una orden y demostrar que no puede existir un segundo certificado con la misma `OrdenId`.

Solución: partir del código de `M02/PROYECTO/2.4`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Una relación uno-a-uno representa un expediente técnico único asociado a una orden: puede no existir todavía, pero cuando existe pertenece a esa orden.

Resultado esperado

Al terminar 2.4, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene `2.4 OK`. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es `2.5`. Se parte del proyecto completo de 2.4; no se vuelve a crear `AceriaData` desde cero.

Código acumulativo completo del estado 2.4

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
```

CSHARP

```

public int OrdenId { get; set; }
public double Espesor { get; set; }
public double Ancho { get; set; }
public double Largo { get; set; }
public decimal Peso { get; set; }
public bool Activa { get; set; } = true;
public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>

```

```

{
    entity.ToTable("Aleaciones");
    entity.HasKey(a => a.Id);
    entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
    entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
    entity.Property(a => a.Descripcion).HasMaxLength(500);
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {

```

```

        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
                OrganismoCertificador = "Laboratorio Aceria"
            },
        };

        context.OrdenesFabricacion.Add(orden);
        context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
        context.SaveChanges();

        var cargada = context.OrdenesFabricacion.Include(o => o.Detalle).Include(o => o.Certificado).Single(o =>

```

```
o.NumeroOrden == "OF-M2-0001");
    global::System.Console.WriteLine($"2.4 OK | Detalle: {cargada.Detalle?.ComposicionQuimica} | Certificado:
{cargada.Certificado?.NumeroCertificado}");
}
}
```

Punto 2.5 - Relaciones muchos a muchos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.5 parte de 2.4 y tiene como objetivo incorporar una entidad intermedia explícita OrdenAleacion con datos propios.

Objetivos de aprendizaje

- Distinguir many-to-many implícito de entidad intermedia explícita.
- Crear OrdenAleacion.
- Configurar dos relaciones uno-a-muchos.
- Definir una clave primaria compuesta.
- Añadir datos de la relación.
- Cargar la relación con Include/ThenInclude.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.5
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. AceriaData.sln está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.4. En 2.5 se introduce exclusivamente el contenido que corresponde a **Relaciones muchos a muchos**. La práctica no reinicia AceriaData ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });

    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

CSHARP

Línea 1	HasKey(x => new { x.OrdenFabricacionId, x.AleacionId }) → define la clave primaria compuesta de la entidad puente.
Línea 2	WithMany(o => o.OrdenesAleaciones) → modela una de las dos relaciones uno-a-muchos que forman el muchos-a-muchos.
Línea 3	DeleteBehavior.Cascade → elimina las filas puente cuando desaparece la orden.

Línea 4	DeleteBehavior.Restrict → evita eliminar una aleación mientras siga referenciada.
Línea 5	CantidadUtilizada → demuestra por qué se usa una entidad puente explícita: la relación tiene datos propios.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_5
dotnet ef database update
```

POWERSHELL

dotnet ef migrations add compara el modelo actual con el snapshot heredado. dotnet ef database update aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa Database.Migrate() para aplicar el historial durante la demostración. El laboratorio puede usar EnsureDeleted() para empezar desde una base limpia, pero **no usa EnsureCreated()**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.5 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Clave duplicada	Se repite el par OrdenFabricacionId/ AleacionId	La PK compuesta impide duplicados.
Borrado de Aleacion falla	DeleteBehavior.Restrict protege referencias	Eliminar primero las relaciones o cambiar conscientemente la estrategia.
Se usa tabla puente implícita	La relación tiene propiedades propias	Usar OrdenAleacion explícita.

Reto resuelto

Reto: Asignar dos aleaciones distintas a una orden, cada una con CantidadUtilizada diferente, y recuperarlas con ThenInclude.

Solución: partir del código de M02/PROYECTO/2.5, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

OrdenAleacion funciona como una hoja de asignación: une dos elementos y además registra datos propios de esa asignación.

Resultado esperado

Al terminar 2.5, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.5 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.6. Se parte del proyecto completo de 2.5; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.5

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
```

```

    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
            entity.Property(e => e.Descripcion).HasMaxLength(250);
            entity.Property(e => e.Activo).HasDefaultValue(true);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);

```

```

        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

```

```

    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
                OrganismoCertificador = "Laboratorio Aceria"
            },
        };

        var aleacion = new Aleacion
        {
            Nombre = "AISI 1045",
           Codigo = "A1045",
            PorcentajeCarbono = 0.45,
            PorcentajeManganeso = 0.75,
            Descripcion = "Acero medio en carbono"
        };
        orden.OrdenesAleaciones.Add(new OrdenAleacion
        {

```

```

        Aleacion = aleacion,
        CantidadUtilizada = 1500.500m,
        EstadoRelacion = "Activa"
    });
    context.OrdenesFabricacion.Add(orden);
    context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
    context.SaveChanges();

    var cargada = context.OrdenesFabricacion.Include(o => o.OrdenesAleaciones).ThenInclude(x =>
x.Aleacion).Single(o => o.NumeroOrden == "OF-M2-0001");
    global::System.Console.WriteLine($"2.5 OK | Aleaciones: {cargada.OrdenesAleaciones.Count} | Primera:
{cargada.OrdenesAleaciones[0].Aleacion.Codigo}");
    }
}

```

Punto 2.6 - Configuración mediante Data Annotations

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.6 parte de 2.5 y tiene como objetivo expresar parte del mapeo con atributos sin perder la configuración acumulada.

Objetivos de aprendizaje

- Aplicar Table, Key, Required y MaxLength.
- Usar ForeignKey en navegaciones.
- Usar Precision para valores decimales.
- Definir una clave compuesta con PrimaryKey.
- Comprender la coexistencia con Fluent API.
- Comprobar metadatos del modelo resultante.

Paso 1: Abrir la solución autónoma del punto

```

cd M02/PROYECTO/2.6
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release

```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.5. En 2.6 se introduce exclusivamente el contenido que corresponde a **Configuración mediante Data Annotations**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }

    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }

    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
}

```

CSHARP

Línea 1

[Table(...)] → configura el nombre de tabla mediante atributo.

Línea 2	[Key] → identifica una clave primaria simple.
Línea 3	[Required] → refuerza la obligatoriedad de una propiedad.
Línea 4	[MaxLength(...)] → declara longitud máxima desde la clase.
Línea 5	[PrimaryKey(...)] → declara correctamente una clave primaria compuesta en EF Core 8.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_6
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia **2.6 OK**. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Se colocan dos [Key]	No representa correctamente una clave compuesta	Usar [PrimaryKey(...)] o HasKey.
Atributos no surten efecto	Fluent API posterior los sobrescribe	Recordar prioridad Fluent API > annotations > convenciones.
Se duplica configuración	Se repite el mismo detalle sin propósito	Mantener annotations como demostración y Fluent donde sea necesario.

Reto resuelto

Reto: Explicar en el modelo final qué configuraciones proceden de annotations y cuáles quedan sobrescritas por Fluent API.

Solución: partir del código de `M02/PROYECTO/2.6`, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Las Data Annotations son indicaciones escritas directamente sobre cada pieza del modelo; viajan con la clase.

Resultado esperado

Al terminar 2.6, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.6 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.7. Se parte del proyecto completo de 2.6; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.6

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
```

```

    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {

```

```

        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(x => x.Id);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes

```

```

{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
            }
        };
    }
}

```

```

        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganizacionesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganizacionesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.6 OK | Tabla por annotations: {entity.GetTableName()} | NumeroOrden
MaxLength: {entity.FindProperty(nameof(OrdenFabricacion.NumeroOrden)).GetMaxLength()}");
}
}

```

Punto 2.7 - Configuración mediante Fluent API

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.7 parte de 2.6 y tiene como objetivo centralizar y hacer explícita la configuración con Fluent API.

Objetivos de aprendizaje

- Comprender la prioridad de Fluent API.
- Configurar propiedades desde OnModelCreating.
- Configurar relaciones con expresiones fuertemente tipadas.
- Usar HasDefaultValue y HasDefaultValueSql.
- Mantener Data Annotations compatibles.
- Reservar claves, índices y filtros para sus puntos específicos.

Paso 1: Abrir la solución autónoma del punto

```

cd M02/PROYECTO/2.7
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release

```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.6. En 2.7 se introduce exclusivamente el contenido que corresponde a **Configuración mediante Fluent API**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.Property(o => o.NumeroOrden)
        .IsRequired()
        .HasMaxLength(50);

    entity.Property(o => o.FechaCreacion)
        .HasDefaultValueSql("GETDATE()");
});
```

CSHARP

Línea 1	modelBuilder.Entity<T>() → selecciona la entidad a configurar.
Línea 2	Property(...) → selecciona una propiedad escalar.
Línea 3	IsRequired() → configura obligatoriedad.
Línea 4	HasMaxLength(...) → configura el tamaño máximo.
Línea 5	HasDefaultValueSql("GETDATE()") → delega el valor por defecto de FechaCreacion en SQL Server.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_7
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.7 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Fluent contradice atributos	Dos configuraciones dan valores distintos	Definir una fuente de verdad consciente; Fluent gana.
Se introduce HasQueryFilter	Se adelantó 2.10	Reservarlo para filtros globales.
Se añaden índices avanzados	Se adelantó 2.9	Mantener 2.7 en sintaxis y configuración del modelo.

Reto resuelto

Reto: Mover una regla de MaxLength desde annotation a Fluent API y comprobar que el modelo resultante conserva la misma longitud.

Solución: partir del código de M02/PROYECTO/2.7, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Fluent API es el plano central de configuración: permite expresar reglas que no caben cómodamente como atributos.

Resultado esperado

Al terminar 2.7, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.7 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.8. Se parte del proyecto completo de 2.7; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.7

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
```

CSHARP

```

public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
}

```

```

public DateTime FechaAsignacion { get; set; } = DateTime.Now;
[Precision(18, 3)]
public decimal CantidadUtilizada { get; set; }
[MaxLength(20)]
public string EstadoRelacion { get; set; } = "Activa";
public OrdenFabricacion Orden { get; set; } = null!;
public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones");
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
            entity.Property(e => e.Descripcion).HasMaxLength(250);
            entity.Property(e => e.Activo).HasDefaultValue(true);
        });
        modelBuilder.Entity<DetalleOrden>(entity =>
        {
            entity.ToTable("DetallesOrden");
            entity.HasKey(d => d.Id);
            entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
            entity.Property(d => d.Notas).HasMaxLength(500);
            entity.HasOne(d => d.Orden)
                .WithOne(o => o.Detalle)
                .HasForeignKey<DetalleOrden>(d => d.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
        });
        modelBuilder.Entity<CertificadoCalidad>(entity =>
        {
            entity.ToTable("CertificadosCalidad");
            entity.HasKey(c => c.Id);
            entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
            entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
            entity.HasOne(c => c.Orden)
                .WithOne(o => o.Certificado)
                .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)

```

```

        .onDelete(DeleteBehavior.Cascade)
        .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .onDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .onDelete(DeleteBehavior.Restrict);
    });
}

public interface IOOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>

```

```

        options
            .UseSqlServer(connectionString, sql =>
            {
                sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                sql.CommandTimeout(60);
            })
            .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                LogLevel.Information)
            .EnableDetailedErrors());

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateScopes = true,
    ValidateOnBuild = true
});

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.7 OK | Fluent API | Estado default:
{entity.FindProperty(nameof(OrdenFabricacion.Estado))?.GetDefaultValue()}");
}
}

```

Punto 2.8 - Claves primarias, alternativas y compuestas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.8 parte de 2.7 y tiene como objetivo formalizar claves simples, alternativas y compuestas.

Objetivos de aprendizaje

- Distinguir clave primaria de clave alternativa.
- Usar HasAlternateKey.
- Mantener la clave compuesta de OrdenAleacion.
- Comprender la unicidad generada por una alternate key.
- Inspeccionar las claves del modelo.
- Preparar el modelo para índices y restricciones.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.8
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.7. En 2.8 se introduce exclusivamente el contenido que corresponde a **Claves primarias, alternativas y compuestas**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasKey(o => o.Id);
    entity.HasAlternateKey(o => o.NumeroOrden)
        .HasName("AK_OrdenesFabricacion_NumeroOrden");
});

modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
});
```

CSHARP

Línea 1	<code>HasKey(...)</code> → configura la clave primaria.
Línea 2	<code>HasAlternateKey(...)</code> → configura una clave candidata adicional con unicidad.
Línea 3	<code>HasName(...)</code> → da un nombre estable a la restricción en SQL Server.
Línea 4	<code>new { x.OrdenFabricacionId, x.AleacionId }</code> → forma una clave compuesta con dos propiedades.
Línea 5	<code>GetKeys()</code> → permite inspeccionar las claves configuradas en metadatos.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

POWERSHELL

```
dotnet ef migrations add M2_2_8
dotnet ef database update
```

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.8 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

POWERSHELL

```
dotnet ef migrations list
```

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Se confunde índice único con alternate key	Tienen finalidades de modelo diferentes	Usar <code>HasAlternateKey</code> para una clave candidata; índices se estudian en 2.9.
PK compuesta incompleta	Falta una de las columnas	Configurar ambas propiedades en <code>HasKey</code> .
Valor alternativo repetido	Viola la restricción UNIQUE	Validar <code>NumeroOrden/Codigo</code> antes de guardar.

Reto resuelto

Reto: Intentar insertar dos órdenes con el mismo `NumeroOrden` y observar que la clave alternativa protege la unicidad.

Solución: partir del código de M02/PROYECTO/2.8, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Las claves son los identificadores y restricciones de identidad del sistema; una clave alternativa es otro identificador candidato, no sólo una ayuda de rendimiento.

Resultado esperado

Al terminar 2.8, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.8 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.9. Se parte del proyecto completo de 2.8; no se vuelve a crear `AceriaData` desde cero.

Código acumulativo completo del estado 2.8

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}
```

```

}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero");
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)

```

```

        .HasForeignKey(x => x.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones");
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });
    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
    });
    }
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);

```

```

    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            }
        }
    }
}

```

```

    }
},
Detalle = new DetalleOrden
{
    ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
    TemperaturaColada = 1550.5,
    Notas = "Colada principal"
},
Certificado = new CertificadoCalidad
{
    NumeroCertificado = "CERT-0001",
    FechaEmision = DateTime.Today,
    OrganismoCertificador = "Laboratorio Aceria"
},
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganizacionesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganizacionesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.8 OK | Alternate keys: {entity.GetKeys().Count()} | PK:
{string.Join(", ", entity.FindPrimaryKey().Properties.Select(x => x.Name))}");
}
}

```

Punto 2.9 - Índices y restricciones

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.9 parte de 2.8 y tiene como objetivo añadir índices simples, compuestos, filtrados y cubrientes, además de restricciones CHECK.

Objetivos de aprendizaje

- Crear índices con HasIndex.
- Configurar índices compuestos.
- Configurar índices filtrados.
- Usar IncludeProperties en SQL Server.
- Añadir restricciones CHECK.
- Verificar que migraciones y modelo permanecen sincronizados.

Paso 1: Abrir la solución autónoma del punto

```

cd M02/PROYECTO/2.9
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release

```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.8. En 2.9 se introduce exclusivamente el contenido que corresponde a **Índices y restricciones**. La práctica no reinicia AceriaData ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```
entity.HasIndex(o => new { o.Cliente, o.FechaCreacion })
    .HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");

entity.HasIndex(o => o.FechaEntrega)
    .HasFilter("[Estado] = 'Pendiente'")
    .HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");

entity.HasIndex(o => o.Estado)
    .IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion })
    .HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
```

CSHARP

Línea 1	HasIndex(...) → crea un índice para las propiedades seleccionadas.
Línea 2	new { o.Cliente, o.FechaCreacion } → define un índice compuesto respetando el orden de columnas.
Línea 3	HasFilter(...) → limita el índice a las filas que cumplen una condición SQL.
Línea 4	IncludeProperties(...) → añade columnas incluidas al índice de SQL Server.
Línea 5	HasCheckConstraint(...) → traslada una regla de integridad al propio motor de base de datos.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_9
dotnet ef database update
```

POWERSHELL

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa `EnsureCreated()`**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.9 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Índice redundante	Se indexan las mismas columnas sin necesidad	Relacionar cada índice con consultas reales.
CHECK falla al migrar	Existen datos incompatibles	Corregir datos antes de aplicar la restricción.
Filtro de índice no coincide con SQL Server	Expresión SQL inválida	Usar sintaxis válida para el proveedor real.

Reto resuelto

Reto: Intentar guardar una plancha con Espesor negativo y comprobar que SQL Server rechaza la operación por CHECK.

Solución: partir del código de M02/PROYECTO/2.9, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Los índices son caminos de acceso y las restricciones son controles de calidad en la propia base de datos.

Resultado esperado

Al terminar 2.9, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.9 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.10. Se parte del proyecto completo de 2.9; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.9

El siguiente archivo es el `Program.cs` real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

```

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }

```

```

public int AleacionId { get; set; }
public DateTime FechaAsignacion { get; set; } = DateTime.Now;
[Precision(18, 3)]
public decimal CantidadUtilizada { get; set; }
[MaxLength(20)]
public string EstadoRelacion { get; set; } = "Activa";
public OrdenFabricacion Orden { get; set; } = null!;
public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'");
            entity.HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
            entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero", t =>
            {
                t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
            });
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
            entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
            entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones", t =>
            {
                t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
                t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
            });
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
            entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
            entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
            entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {

```

```

        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
    });
    }
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AcriaDbContext _context;
    public OrdenRepositorio(AcriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes

```

```

{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                {
                    sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                    sql.CommandTimeout(60);
                })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOordenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
            }
        }
    }
}

```

```

        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganizacionesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganizacionesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var entity = context.Model.FindEntityType(typeof(OrdenFabricacion));
global::System.Console.WriteLine($"2.9 OK | Índices: {string.Join(", ", entity.GetIndexes().Select(i =>
i.GetDatabaseName()))}");
}
}

```

Punto 2.10 - Filtros globales de consulta

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.10 parte de 2.9 y tiene como objetivo añadir HasQueryFilter sin introducir todavía las propiedades de Soft Delete.

Objetivos de aprendizaje

- Configurar HasQueryFilter.
- Filtrar órdenes canceladas.
- Filtrar planchas inactivas.
- Filtrar relaciones OrdenAleacion no activas.
- Usar IgnoreQueryFilters de forma explícita.
- Demostrar que el registro sigue existiendo en la base.

Paso 1: Abrir la solución autónoma del punto

```

cd M02/PROYECTO/2.10
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release

```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.9. En 2.10 se introduce exclusivamente el contenido que corresponde a **Filtros globales de consulta**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

```

modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasQueryFilter(o => o.Estado != "Cancelada");
}

```

CSHARP

```
});

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasQueryFilter(p => p.Activa);
});

var visibles = context.OrdenesFabricacion.Count();
var todas = context.OrdenesFabricacion
    .IgnoreQueryFilters()
    .Count();
```

Línea 1	HasQueryFilter(...) → añade una condición transversal a todas las consultas normales de la entidad.
Línea 2	o.Estado != "Cancelada" → excluye las órdenes canceladas sin borrarlas.
Línea 3	p.Activa → limita las planchas visibles a las activas.
Línea 4	IgnoreQueryFilters() → desactiva expresamente los filtros para una consulta concreta.
Línea 5	Count() → permite comparar de manera reproducible la vista filtrada con la vista completa.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

```
dotnet ef migrations add M2_2_10
dotnet ef database update
```

POWERSHELL

dotnet ef migrations add compara el modelo actual con el snapshot heredado. dotnet ef database update aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa Database.Migrate() para aplicar el historial durante la demostración. El laboratorio puede usar EnsureDeleted() para empezar desde una base limpia, pero **no usa EnsureCreated()**, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

POWERSHELL

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.10 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

```
dotnet ef migrations list
```

POWERSHELL

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
Registro filtrado parece borrado	HasQueryFilter lo oculta	Comprobar con IgnoreQueryFilters.

Error	Causa	Solución
IgnoreQueryFilters se usa indiscriminadamente	Se desactiva la regla transversal	Reservarlo para diagnósticos/administración.
Se añade IsDeleted	Se adelanta 2.11	En 2.10 usar sólo filtros de negocio ya existentes.

Reto resuelto

Reto: Insertar una orden Cancelada y comparar Count() con Count() después de IgnoreQueryFilters().

Solución: partir del código de M02/PROYECTO/2.10, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Un filtro global actúa como una regla automática de visibilidad que EF Core adjunta a cada consulta normal.

Resultado esperado

Al terminar 2.10, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.10 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.11. Se parte del proyecto completo de 2.10; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.10

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

CSHARP

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
```

```

    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
}

```

```

public OrdenFabricacion Orden { get; set; } = null!;
public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] =
'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
            entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
            entity.HasQueryFilter(o => o.Estado != "Cancelada");
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero", t =>
            {
                t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
            });
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
            entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
            entity.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
            entity.HasQueryFilter(x => x.Activa);
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones", t =>
            {
                t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND
[PorcentajeCarbono] <= 2");
                t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND
[PorcentajeManganeso] <= 5");
            });
            entity.HasKey(a => a.Id);
            entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
            entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
            entity.Property(a => a.Descripcion).HasMaxLength(500);
            entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
            entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
            entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        });
        modelBuilder.Entity<EstadoOrden>(entity =>
        {
            entity.ToTable("EstadosOrden");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
            entity.Property(e => e.Descripcion).HasMaxLength(250);
        });
    }
}

```

```

        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<CertificadoCalidad>(entity =>
    {
        entity.ToTable("CertificadosCalidad");
        entity.HasKey(c => c.Id);
        entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
        entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
        entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
        entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
        entity.HasOne(c => c.Orden)
            .WithOne(o => o.Certificado)
            .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
    });

    modelBuilder.Entity<OrdenAleacion>(entity =>
    {
        entity.ToTable("OrdenesAleaciones");
        entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
        entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
        entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.OrdenesAleaciones)
            .HasForeignKey(x => x.OrdenFabricacionId)
            .OnDelete(DeleteBehavior.Cascade);
        entity.HasOne(x => x.Aleacion)
            .WithMany(a => a.OrdenesAleaciones)
            .HasForeignKey(x => x.AleacionId)
            .OnDelete(DeleteBehavior.Restrict);
        entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
        entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'");
        entity.HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
        entity.HasQueryFilter(x => x.EstadoRelacion == "Activa");
    });
}

public interface IOOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    int Guardar();
}

public sealed class OrdenRepositorio : IOOrdenRepositorio
{
    private readonly AcriaDbContext _context;
    public OrdenRepositorio(AcriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;

```

```

private readonly AceriaDbContext _context;
public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)
{
    _repositorio = repositorio;
    _context = context;
}

public string ObtenerResumen(int ordenId)
{
    var orden = _repositorio.ObtenerPorId(ordenId);
    if (orden is null) return $"Orden {ordenId} no encontrada";
    var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
    return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
}

}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
        var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
        context.Database.EnsureDeleted();
        context.Database.Migrate();

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "OF-M2-0001",
            Cliente = "Constructora del Norte",
            FechaCreacion = DateTime.UtcNow,
            FechaEntrega = DateTime.Today.AddDays(14),
            Estado = "Pendiente",
            Observaciones = "Orden de validación M2",
            Planchas =
            {
                new PlanchaAcero
                {
                    Espesor = 10.5,
                    Ancho = 1500,
                    Largo = 3000,
                    Peso = 371.250m,
                    Activa = true,
                }
            },
            Detalle = new DetalleOrden
            {
                ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
                TemperaturaColada = 1550.5,
                Notas = "Colada principal"
            },
            Certificado = new CertificadoCalidad
            {
                NumeroCertificado = "CERT-0001",
                FechaEmision = DateTime.Today,
                OrganismoCertificador = "Laboratorio Aceria"
            }
        }
    }
}

```

```

    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrganosAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrganosFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true });
context.SaveChanges();

var cancelada = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-CANCELADA",
    Cliente = "Cliente Histórico",
    FechaCreacion = DateTime.UtcNow,
    Estado = "Cancelada"
};
context.OrganosFabricacion.Add(cancelada);
context.SaveChanges();
var visibles = context.OrganosFabricacion.Count();
var todas = context.OrganosFabricacion.IgnoreQueryFilters().Count();
global::System.Console.WriteLine($"2.10 OK | Visibles: {visibles} | Sin filtro: {todas}");
}
}

```

Punto 2.11 - Soft Delete

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL que trabajan con .NET 8, EF Core 8 y SQL Server LocalDB.

Proyecto: El estado 2.11 parte de 2.10 y tiene como objetivo añadir IsDeleted y DeletedAt, combinar filtros y restaurar registros.

Objetivos de aprendizaje

- Implementar borrado lógico.
- Registrar DeletedAt.
- Combinar filtros de negocio y borrado lógico.
- Usar IgnoreQueryFilters para restaurar.
- Mantener coherencia en entidades relacionadas.
- Distinguir borrado lógico de borrado físico.

Paso 1: Abrir la solución autónoma del punto

```

cd M02/PROYECTO/2.11
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release

```

POWERSHELL

No se abre una solución global situada en la raíz del repositorio. `AceriaData.sln` está dentro de la carpeta del punto y referencia únicamente los proyectos de ese estado.

Paso 2: Identificar el cambio respecto al estado anterior

El proyecto conserva todo lo terminado en 2.10. En 2.11 se introduce exclusivamente el contenido que corresponde a **Soft Delete**. La práctica no reinicia `AceriaData` ni crea un ejemplo paralelo.

Paso 3: Implementar y comprender la configuración principal

CSHARP

```
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }

entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");

var id = orden.Id;
orden.IsDeleted = true;
orden.DeletedAt = DateTime.UtcNow;
context.SaveChanges();

var restaurable = context.OrdenesFabricacion
    .IgnoreQueryFilters()
    .Single(o => o.Id == id);

restaurable.IsDeleted = false;
restaurable.DeletedAt = null;
context.SaveChanges();
```

Línea 1	IsDeleted → marca el estado lógico de eliminación.
Línea 2	DeletedAt → registra el instante de borrado lógico.
Línea 3	!o.IsDeleted → incorpora la condición de Soft Delete al filtro global.
Línea 4	IgnoreQueryFilters() → permite recuperar una fila eliminada para administrarla.
Línea 5	restaurable.IsDeleted = false → restaura la visibilidad normal de la entidad.

Paso 4: Generar y aplicar la migración acumulativa

El repositorio contiene una migración real generada para este estado. Si se reproduce el ejercicio desde el estado anterior, los comandos son:

POWERSHELL

```
dotnet ef migrations add M2_2_11
dotnet ef database update
```

`dotnet ef migrations add` compara el modelo actual con el snapshot heredado. `dotnet ef database update` aplica únicamente los cambios pendientes.

En los estados ejecutables del curso se usa `Database.Migrate()` para aplicar el historial durante la demostración. El laboratorio puede usar `EnsureDeleted()` para empezar desde una base limpia, pero **no usa** `EnsureCreated()`, porque el esquema está gobernado por Migrations.

Paso 5: Compilar y ejecutar el estado

POWERSHELL

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet run --project AceriaData.Console.csproj --configuration Release
```

Resultado esperado: la ejecución termina sin excepción y contiene la evidencia 2.11 OK. GitHub Actions repite esta compilación y ejecución sobre Windows y SQL Server LocalDB.

Paso 6: Verificar el estado acumulativo

Además de ejecutar el ejemplo, conviene revisar el modelo y la base:

POWERSHELL

```
dotnet ef migrations list
```

Cuando el punto modifica el esquema, la lista debe contener la migración acumulativa correspondiente. En SQL Server Object Explorer se puede comprobar que las tablas, claves, relaciones, índices o restricciones coinciden con el modelo del punto.

Errores comunes

Error	Causa	Solución
No se puede restaurar	La consulta normal oculta la fila	Cargar con IgnoreQueryFilters.
Se llama Remove por error	Se ejecuta borrado físico	Para Soft Delete modificar IsDeleted/DeletedAt.
Dependientes aparecen incoherentes	Filtros de relaciones no están alineados	Aplicar filtros consistentes a dependientes requeridos.

Reto resuelto

Reto: Eliminar lógicamente una orden, comprobar que desaparece de la consulta normal, recuperarla con IgnoreQueryFilters y restaurarla.

Solución: partir del código de M02/PROYECTO/2.11, realizar únicamente el cambio descrito y volver a ejecutar build, migraciones y el programa. El reto no introduce conceptos reservados a un punto posterior.

Analogía final

Soft Delete es archivar una orden sin destruir su expediente: deja de aparecer en la operativa diaria pero puede recuperarse.

Resultado esperado

Al terminar 2.11, la solución local compila, el proyecto se ejecuta, el esquema se actualiza mediante Migrations cuando corresponde y la salida contiene 2.11 OK. El código conserva todos los cambios válidos de los puntos anteriores.

Conexión con el siguiente punto

El siguiente estado es 2.12. Se parte del proyecto completo de 2.11; no se vuelve a crear AceriaData desde cero.

Código acumulativo completo del estado 2.11

El siguiente archivo es el Program.cs real del estado validable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
```

CSHARP

```

}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]

```

```

    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
            entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
            entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
            entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
            entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
            entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
            entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");
        });
        modelBuilder.Entity<PlanchaAcero>(entity =>
        {
            entity.ToTable("PlanchasAcero", t =>
            {
                t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
                t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
            });
            entity.HasKey(x => x.Id);
            entity.Property(x => x.Peso).HasPrecision(18, 3);
            entity.Property(x => x.Activa).HasDefaultValue(true);
            entity.HasOne(x => x.Orden)
                .WithMany(o => o.Planchas)
                .HasForeignKey(x => x.OrdenId)
                .OnDelete(DeleteBehavior.Cascade)
                .IsRequired();
            entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
            entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
            entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);
        });
        modelBuilder.Entity<Aleacion>(entity =>
        {
            entity.ToTable("Aleaciones", t =>
            {
                t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");
                t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");
            });
            entity.HasKey(a => a.Id);
        });
    }
}

```

```

entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
entity.Property(a => a.Descripcion).HasMaxLength(500);
entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
entity.HasQueryFilter(a => !a.IsDeleted);
});
modelBuilder.Entity<EstadoOrden>(entity =>
{
    entity.ToTable("EstadosOrden");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
    entity.Property(e => e.Descripcion).HasMaxLength(250);
    entity.Property(e => e.Activo).HasDefaultValue(true);
    entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);
});
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("DetallesOrden");
    entity.HasKey(d => d.Id);
    entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
    entity.Property(d => d.Notas).HasMaxLength(500);
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");
});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");
});

modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
    entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'");
    entity.HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
    entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);
});
}
}

public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    void EliminarLogicamente(int id);
    void Restaurar(int id);

    int Guardar();
}

public sealed class OrdenRepositorio : IOrdenRepositorio
{

```

```

private readonly AceriaDbContext _context;
public OrdenRepositorio(AceriaDbContext context) => _context = context;

public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();
public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o =>
o.NumeroOrden == numeroOrden);
public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
public void EliminarLogicamente(int id)
{
    var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
    if (orden is null) return;
    orden.IsDeleted = true;
    orden.DeletedAt = DateTime.UtcNow;
}

public void Restaurar(int id)
{
    var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);
    if (orden is null) return;
    orden.IsDeleted = false;
    orden.DeletedAt = null;
}

public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOordenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOordenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
                    LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOordenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });
    }
}

```

```

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    IsDeleted = false,
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden.OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context.OrdenesFabricacion.Add(orden);
context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo =
true, IsDeleted = false });
context.SaveChanges();

var id = orden.Id;
orden.IsDeleted = true;
orden.DeletedAt = DateTime.UtcNow;
context.SaveChanges();
var visibles = context.OrdenesFabricacion.Count();
var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();
var restaurable = context.OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id);
restaurable.IsDeleted = false;
restaurable.DeletedAt = null;
context.SaveChanges();
global::System.Console.WriteLine($"2.11 OK | Tras borrar visibles: {visibles} | Totales: {todas} |
Restaurada: {context.OrdenesFabricacion.Any(o => o.Id == id)}");
}
}

```

Punto 2.12 - Clean Architecture y Arquitectura Hexagonal

Audiencia: Desarrolladores que ya han completado el modelado acumulativo 2.1-2.11.

Proyecto: 2.12 parte de 2.11 y refactorizar el mismo sistema en Domain, Application, Infraestructure y Console. El refactor cambia la organización del código, no el dominio ni el objetivo de persistencia.

Objetivos de aprendizaje

- Separar dominio y persistencia.
- Mantener EF Core fuera de Domain y Application.
- Definir puertos de repositorio y unidad de trabajo.
- Implementar adaptadores en Infrastructure.
- Registrar dependencias desde una extensión de IServiceCollection.
- Conservar el historial de migraciones y el comportamiento del modelo.

Paso 1: Abrir la solución multiproyecto

```
cd M02/PROYECTO/2.12
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

POWERSHELL

La solución contiene cuatro proyectos:

```
src/
├─ AceriaData.Domain
├─ AceriaData.Application
├─ AceriaData.Infrastructure
└─ AceriaData.Console
```

TEXT

Paso 2: Mantener el dominio independiente de EF Core

```
namespace AceriaData.Domain.Entities;

public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public string? Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public stringCodigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    public string? Descripcion { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class EstadoOrden
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Estado { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    public string Observaciones { get; set; }
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

```

public int Id { get; set; }
public string Nombre { get; set; } = string.Empty;
public string Descripcion { get; set; } = string.Empty;
public bool Activo { get; set; } = true;
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class CertificadoCalidad
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    public string OrganismoCertificador { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    public decimal CantidadUtilizada { get; set; }
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

```

Las entidades están en Domain y no necesitan DbContext, DbSet, IEntityTypeConfiguration ni referencias a paquetes de EF Core.

Paso 3: Definir los puertos de aplicación

```

using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}

public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    int Guardar();
}

```

CSHARP

Línea 1 IOrdenRepositorio → expresa operaciones que necesita la aplicación sin exponer EF Core.

Línea 2 IUnidadDeTrabajo → coordina repositorios y el guardado como abstracción de aplicación.

Paso 4: Crear un caso de uso

```

using AceriaData.Application.Interfaces;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.UseCases;

public sealed class CrearOrdenUseCase

```

CSHARP

```

{
    private readonly IUnidadDeTrabajo _unidad;
    public CrearOrdenUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public int Ejecutar(string numeroOrden, string cliente)
    {
        _unidad Ordenes.Agregar(new OrdenFabricacion
        {
            NumeroOrden = numeroOrden,
            Cliente = cliente,
            FechaCreacion = DateTime.UtcNow,
            Estado = "Pendiente"
        });
        return _unidad.Guardar();
    }
}

```

El caso de uso depende de interfaces de Application y de entidades de Domain. No referencia Microsoft.EntityFrameworkCore.

Paso 5: Colocar EF Core en Infrastructure

CSHARP

```

using AceriaData.Domain.Entities;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Infrastructure.Persistence;

public sealed class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);
    }
}

```

AceriaDbContext vive en Infrastructure y aplica configuraciones mediante ApplyConfigurationsFromAssembly. Las migraciones también se encuentran en este proyecto.

Paso 6: Registrar el adaptador de infraestructura

CSHARP

```

using AceriaData.Application.Interfaces;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;

namespace AceriaData.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddAceriaInfrastructure(this IServiceCollection services, string connectionString)
    {
        services.AddDbContext<AceriaDbContext>(o => o.UseSqlServer(connectionString));
        services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
        return services;
    }
}

```

La extensión AddAceriaInfrastructure concentra el registro de SQL Server, repositorios y unidad de trabajo.

Paso 7: Ejecutar desde la capa de entrada

CSHARP

```
using AceriaData.Application.UseCases;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var cs = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(cs);
services.AddScoped<CrearOrdenUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions { ValidateOnBuild = true,
    ValidateScopes = true });
using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var crear = scope.ServiceProvider.GetRequiredService<CrearOrdenUseCase>();
crear.Ejecutar("0F-M2-HEX-0001", "Cliente Arquitectura");
var orden = context.OrdenesFabricacion.Single();
Console.WriteLine($"2.12 OK | {orden.NumeroOrden} | {orden.Cliente}");
```

La consola construye el contenedor, resuelve el caso de uso y ejecuta la aplicación. El resultado esperado contiene 2.12 OK.

Paso 8: Validar migraciones desde la nueva ubicación

POWERSHELL

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El historial de migraciones de 2.1-2.11 se conserva y se añade el estado de arquitectura 2.12.

Errores comunes

Error	Causa	Solución
Domain referencia EF Core	Se movieron entidades junto con tipos de persistencia	Mantener Domain como clases de negocio sin paquete EF Core.
Application usa DbContext	Se filtró una dependencia de infraestructura	Depender de IOrdenRepositorio/IUnidadDeTrabajo.
EF Tools no encuentra el contexto	Startup o project incorrectos	Indicar --project Infrastructure --startup-project Console.
La consola no resuelve servicios	Falta AddAceriaInfrastructure	Registrar la infraestructura antes de BuildServiceProvider.

Reto resuelto

Reto: Crear una orden desde CrearOrdenUseCase y comprobar que Application no referencia Microsoft.EntityFrameworkCore.

Solución: revisar los .csproj: Domain y Application no contienen referencias a Microsoft.EntityFrameworkCore; Infrastructure sí contiene Microsoft.EntityFrameworkCore.SqlServer y Design. Ejecutar después la solución y comprobar el marcador 2.12 OK.

Analogía final

Clean Architecture separa el conocimiento del negocio de la tecnología de persistencia: EF Core pasa a ser un adaptador reemplazable alrededor del núcleo.

Resultado esperado

M2 termina con una solución multiproyecto compilable y ejecutable, el mismo modelo acumulado, EF Core aislado en Infrastructure y una aplicación que accede a persistencia mediante puertos definidos fuera de la infraestructura.

Conexión con el siguiente módulo

M03/PROYECTO0/3.1 deberá partir de este estado completo de 2.12.